

CI/CD Tooling & Practices - Part 1

WELCOME TO CLASS 7!



Goals of the day



Software Development



Jenkins



Test Automation



Photo by [David Becker](#) on [Unsplash](#)

©Maja Bjeletic@unsplash

Agenda

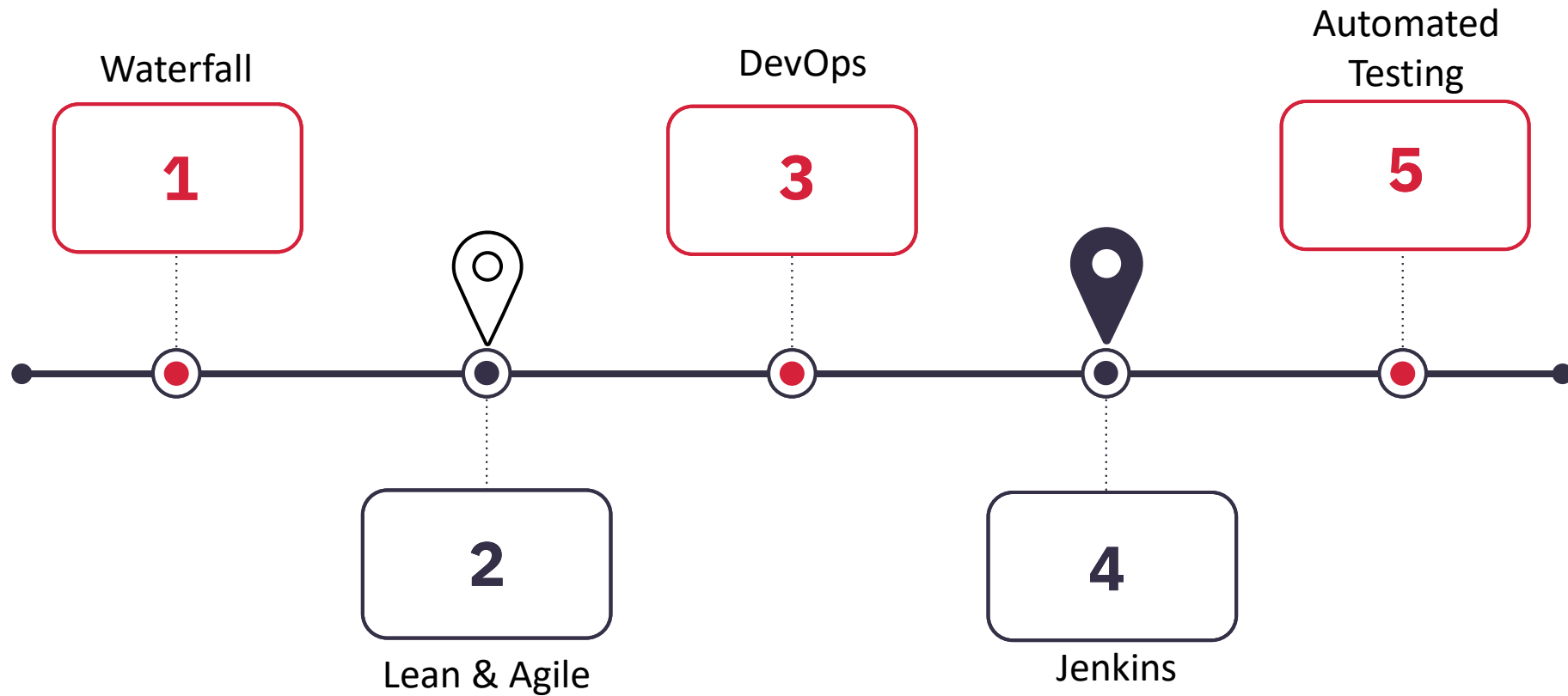
- Software Development – then and now
- **Break**
- Flow
- Introduction to Jenkins
- Testing Automation

Photo by [David Becker](#) on [Unsplash](#)

Review

- What have you learned so far about automation for CloudOps?
- What are the benefits you see?
- Are there any pitfalls which concern you?
- Why do we provision and configure compute resources? What purpose do they achieve in supporting the enterprise?
- What do you know about the development and delivery of software?
- Brief Q & A

Journey Map



Waterfall, Lean & Agile

*“Software development is a young profession,
and we are still learning the techniques and
building the tools to do it effectively.”*

– Martin Fowler

* <https://martinfowler.com>



Photo: Christopher Ferguson

What is Software Development?

- Software development refers to the design, documentation, programming, testing, and ongoing maintenance of a software deliverable. *
- It is often an undervalued, oversimplified and misunderstood process, which poses significant risk and unexpected costs for organizations that get it wrong.
- Yet if you are part of a software delivery, the process can feel at times compromised, rushed and unstable.



Image: Atlassian

* <https://www.atlassian.com/software-development>

Software Engineering

- Software Engineering has been around for half a century.
- It was coined and popularised by Margaret Hamilton, who led the Software Engineering Division of the MIT Instrumentation Laboratory, which in 1961 contracted with NASA to develop the Apollo program's guidance system. *
- This photo shows Margaret with a printout of the Apollo program's guidance system code.

Software eventually and necessarily gained the same respect as any other discipline.

- Margaret H. Hamilton



Credit: Courtesy MIT Museum

* <https://solarsystem.nasa.gov/people/320/margaret-hamilton/>

Software Engineering (con't)

"The Roman bridges of antiquity were very inefficient structures. By modern standards, they used too much stone, and as a result, far too much labor to build. Over the years we have learned to build bridges more efficiently, using fewer materials and less labor to perform the same task."

-- Tom Clancy, The Sum of All Fears*

If we are building software with 'too much stone' today, what will this discipline look like with 'fewer materials and less labour'?



Viaduc de Millau, Millau, France
21stC AD

*<https://www.csus.edu/indiv/v/velianitis/161/chaosreport.pdf>

What is Waterfall?

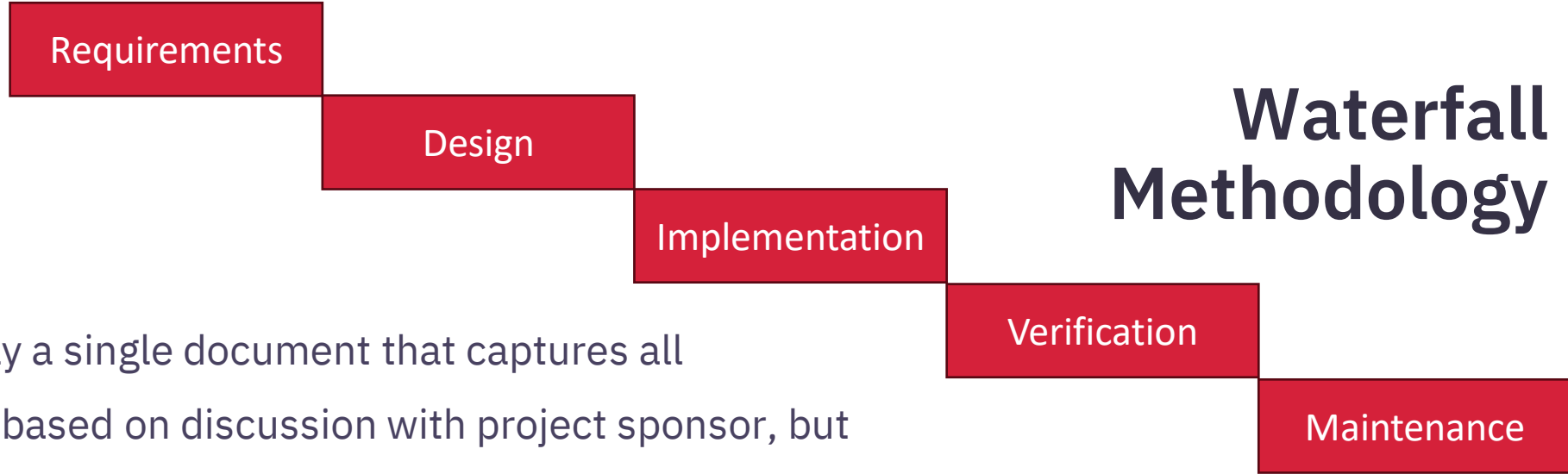
- Not the white-knuckle ride in this image, it is a project management methodology where each phase of the delivery must be complete before the next stage can begin. *
- The success of this approach is dependent on significant work up front to clearly document every scenario and eventuality before development starts. **
- Measure twice, cut once.



Image: DALL-E

* <https://www.forbes.com/advisor/business/what-is-waterfall-methodology/>

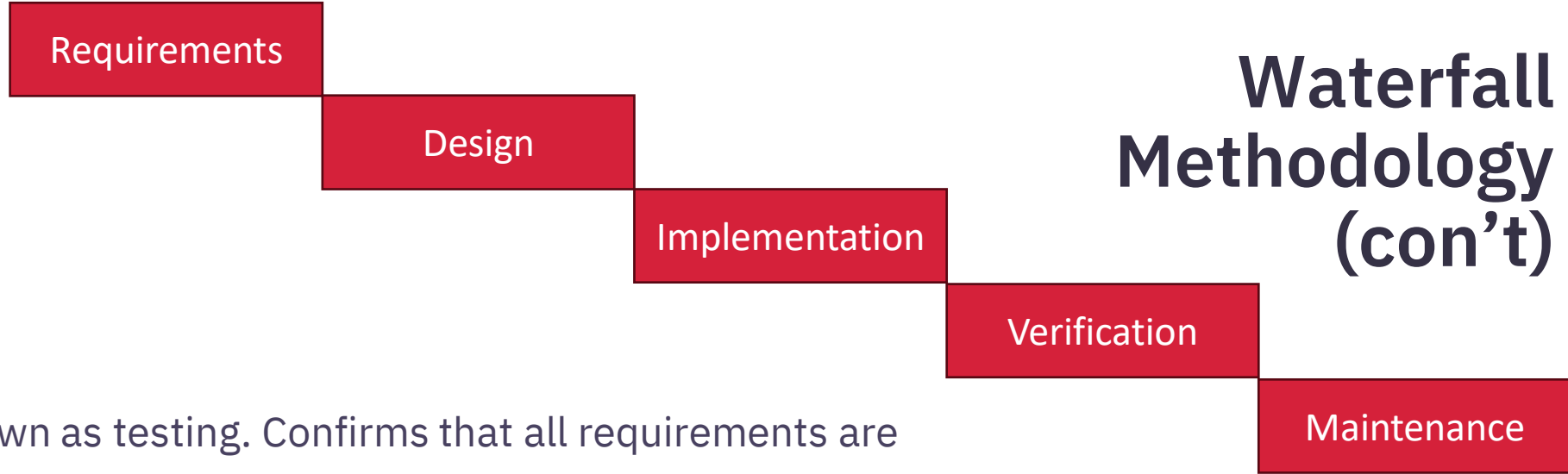
** <https://business.adobe.com/blog/basics/waterfall#advantages-of-the-waterfall-methodology>



- **Requirements:** usually a single document that captures all requirements upfront based on discussion with project sponsor, but with no developer or tester involvement.
- **Design:** first create high level solutions based on the requirements, which are subsequently converted into low level designs as detailed documentation.
- **Implementation:** based on the low-level design, start developing the solution. Issues uncovered at this point can mean reopening the design phase.

Image: DALL-E

* <https://www.forbes.com/advisor/business/what-is-waterfall-methodology/>



- **Verification:** also known as testing. Confirms that all requirements are satisfied, with defects collated and sent back into implementation phase. Typically, there are several test cycles, where all tests are executed, then all defects are sent back to implementation. Creates long gaps between development and fixing.
- **Maintenance:** once delivered, the application needs patching and fixing in the event that errors are found after Go Live.

Image: DALL-E

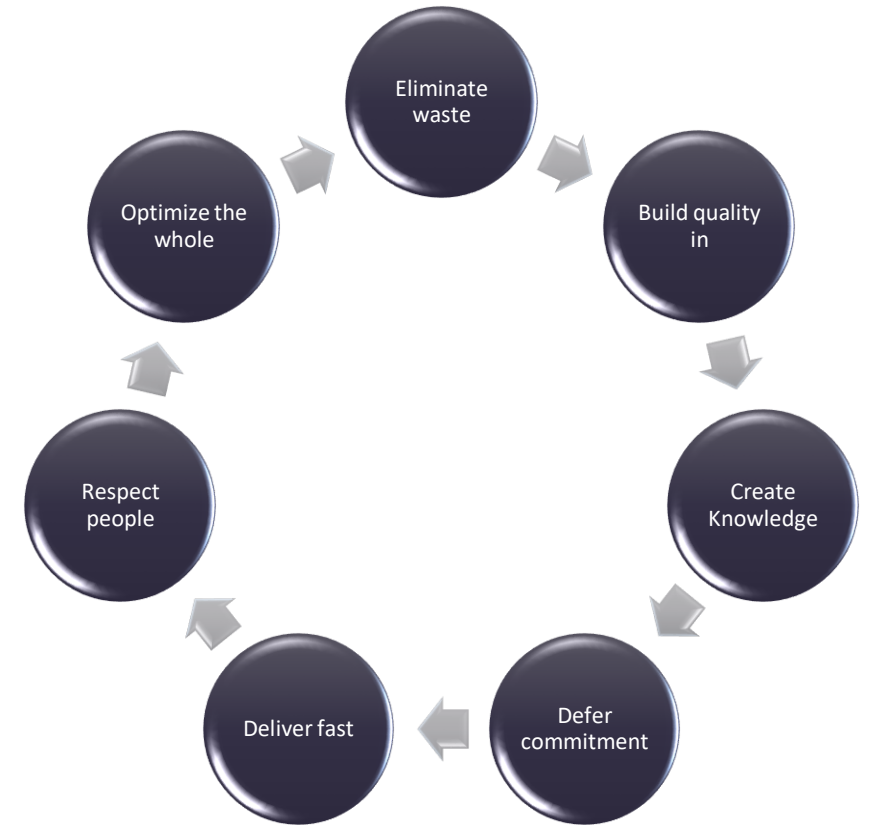
Lean

Originating in 2003 with release of the Lean Software Development book.

This was greatly influenced by the Lean manufacturing approach popularized at Toyota.

The 7 principles align with CI/CD practices, including short lived branches from trunk, test automation, increase flow, learning and experimentation and others.

Lean management also highlights benefits of lightweight change approvals, which is beneficial to DevOps efficiency. *



* Accelerate: The Science of Lean Software and DevOps Building and Scaling High Performing Technology Organizations; Forsgren, N., Humble, J., Kim, G. IT Revolution Press, 2018

Agile

The term Agile has become synonymous with Software Development.

Agile Development is a set of methods and practices where solutions evolve through collaboration between self-organizing, cross-functional teams. *

As an approach, it is highly collaborative and is focused on incremental delivery and iterative steps towards delivery.

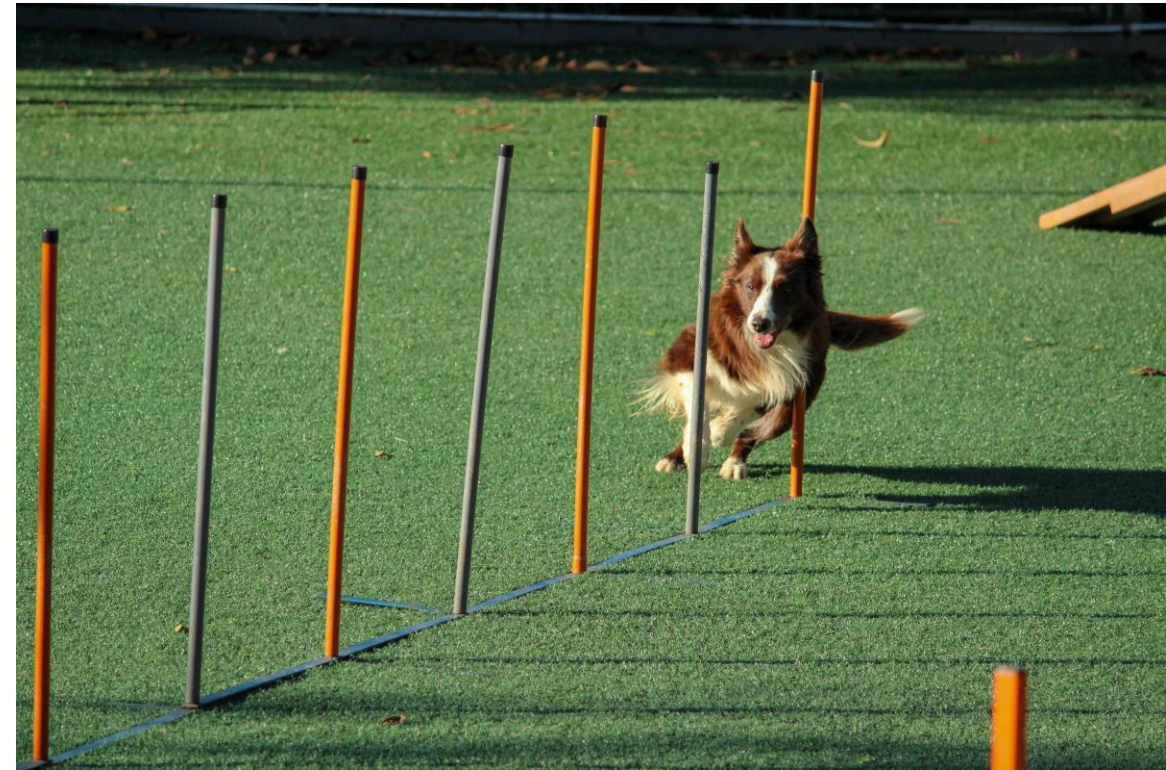


Photo by [Murilo Viviani](#) on [Unsplash](#)

Agile Manifesto

The Agile manifesto was created in 2001 by some of the great software thinkers of the time, representing different methodologies of the time including among others Extreme programming, Feature-Driven Development & Pragmatic Programming

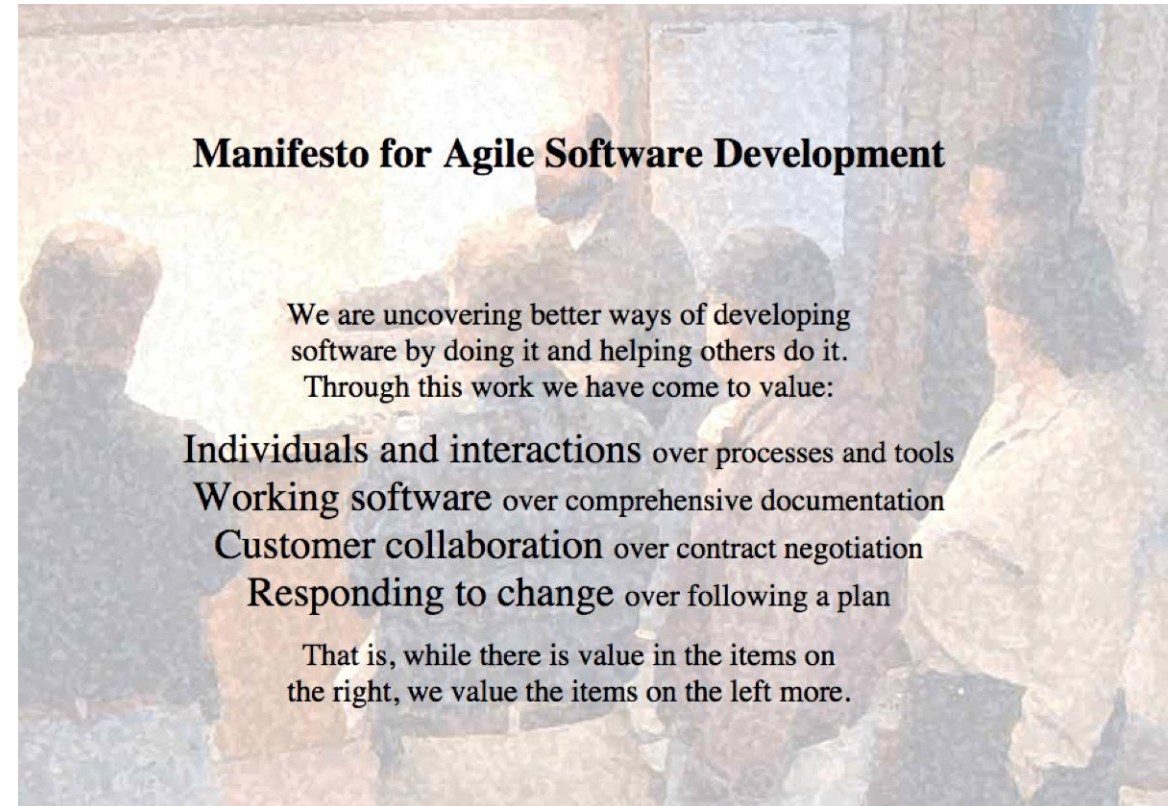


Photo by [Murilo Viviani](#) on [Unsplash](#)
This Photo by Unknown Author is licensed under [CC BY-SA](#)

Agile Principles

1	Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.	7	Working software is the primary measure of progress.
2	Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.	8	Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
3	Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.	9	Continuous attention to technical excellence and good design enhances agility.
4	Business people and developers must work together daily throughout the project.	10	Simplicity—the art of maximizing the amount of work not done—is essential.
5	Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.	11	The best architectures, requirements, and designs emerge from self-organizing teams.
6	The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.	12	At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Compare

- **How do Agile & Waterfall compare?**
- **Which type of project would you like to work on?**
- **Which approach aligns best with DevOps?**
- **Any other comments/feedback on these two approaches?**

* <https://www.forbes.com/advisor/business/what-is-waterfall-methodology/>

DevOps Delivery

Penny Flow

Penny Flow is an experiment to show how different delivery methodologies impact the speed at which software is delivered.

1. Break into teams of 4-5 people, with 20 coins for each team.
2. Roles are timekeeper, BA, Dev, Tester and Ops person. Timekeeper role can be shared amongst other roles if only 4.
3. Process the stack of coins through the entire team. First run will be a batch size of 20 coins. Second will have a batch of 10 coins. Last run will have a batch size of 3.



Penny Flow (con't)

Complete a table like this one to record performance of the value chain based on batch size.

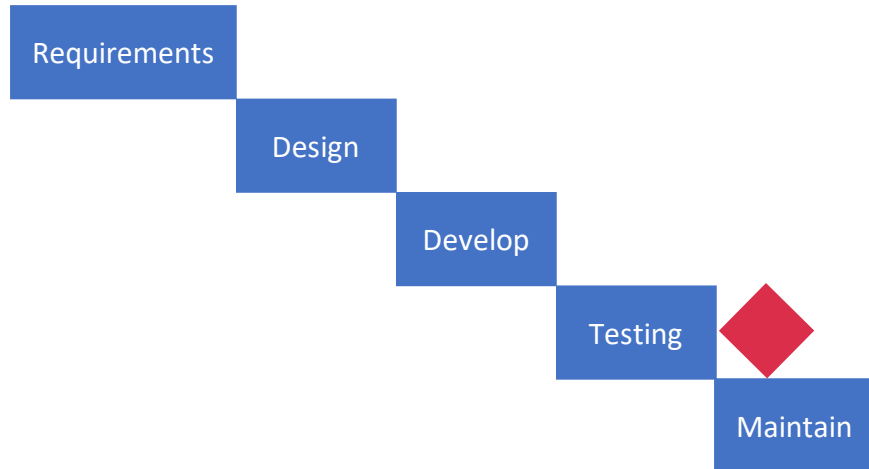
Batch Size	BA	Dev	Tester	Ops	Total Time
20					
10					
3					

Within your team have a short retrospective on the findings of your experiment.

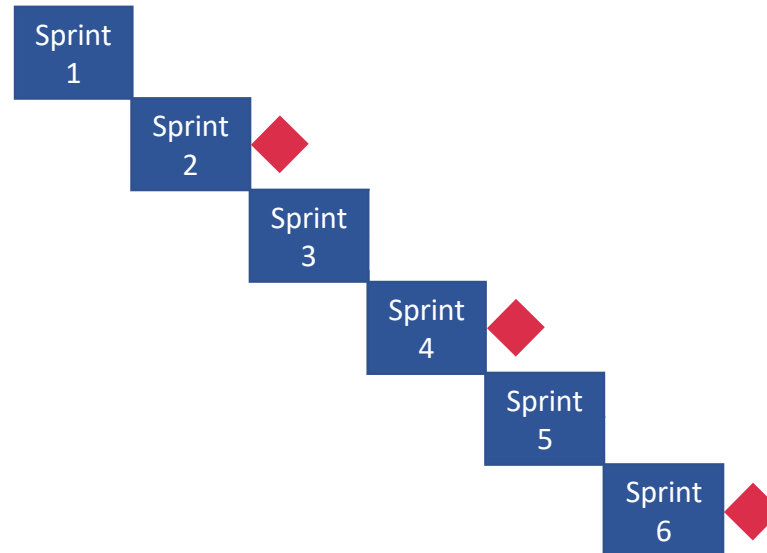
- Did you expect the outcomes you observed?
- How does this impact your thinking on waterfall, lean and agile?

Software Delivery Models

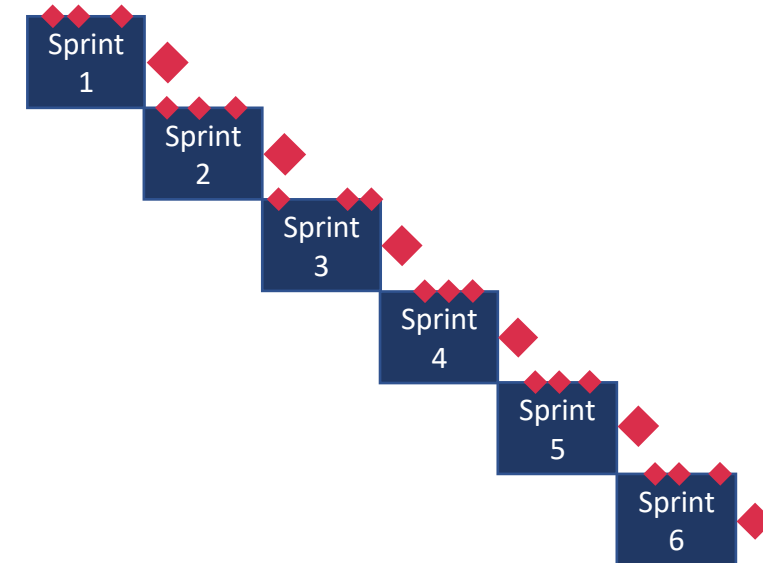
Waterfall



Agile



DevOps



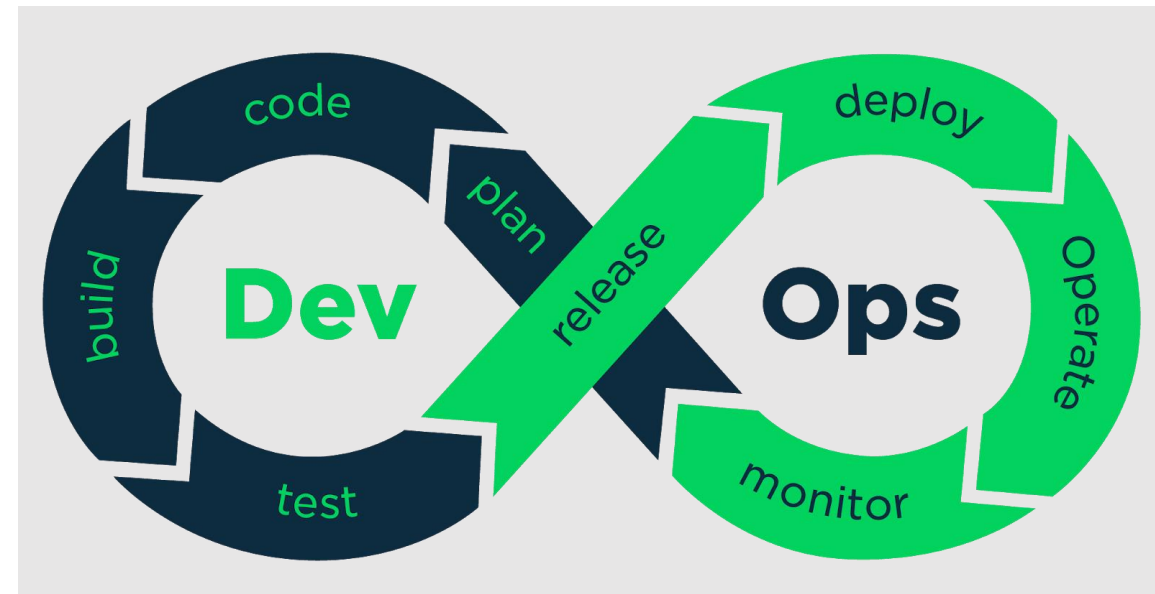
Continuous Everything

Smaller batch sizes and more frequent delivery increase the flow of work and delivery of value to the business.

DevOps delivery is underpinned by Continuous Integration, Continuous Deployment and Continuous Delivery.

Continuous Testing is also part of this approach.

The 8 practices of DevOps are represented in this diagram.



[This Photo](#) by Unknown Author is licensed under [CC BY-SA-NC](#)

Continuous Integration

Continuous Integration is a software development practice where members of a team integrate their work frequently, usually each person integrates at least daily – leading to multiple integrations per day. Each integration is verified by an automated build (including test) to detect integration errors as quickly as possible. Many teams find that this approach leads to significantly reduced integration problems and allows a team to develop cohesive software more rapidly.

<https://martinfowler.com/articles/continuousIntegration.html>

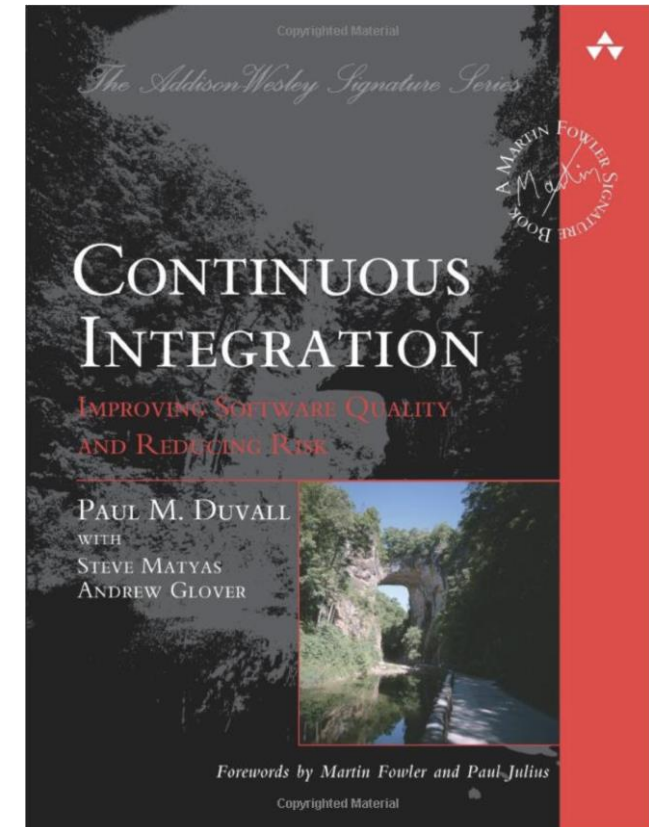


Martin Fowler

Continuous Integration (con't)

To realise Fowler's vision, you need:

- Source code management system
- Agreed branching strategy
- Build script executable from command line
- Automated testing
- Clear approach for managing database changes
- Team culture on broken builds
- Regular commits from all team members
- Continuous improvement mindset



Continuous Delivery

Continuous delivery is an approach where teams release quality products frequently and predictably from source code repository to production in an automated fashion. *

* <https://www.atlassian.com/continuous-delivery>

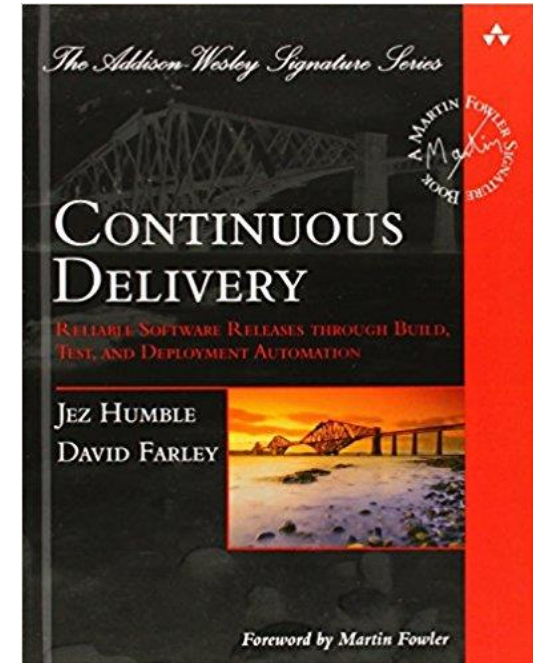


This Photo by Unknown Author is
licensed under CC BY-SA

Continuous Delivery (con't)

This is achieved through the following practices:

- Keep everything in version control
- Build once, deploy many times
- Deploying to production like environment early in the development lifecycle
- Automated environment configuration management
- Changes trigger feedback quickly, which is acted upon
- Done means released
- Approval required to promote code to Production.



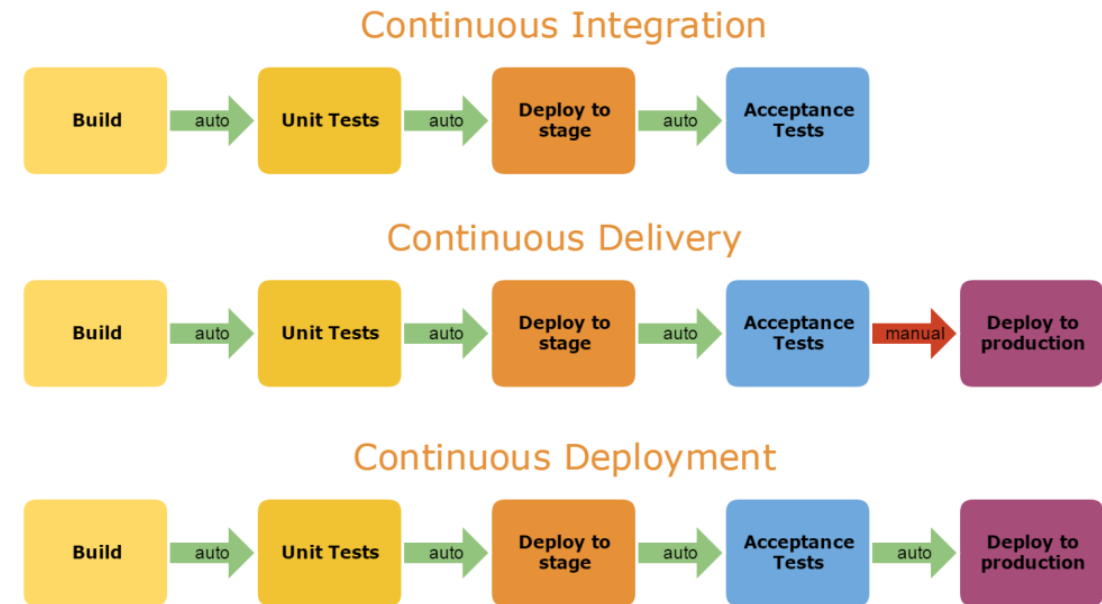
This Photo by Unknown Author is licensed under
CC BY-NC-ND

Continuous Deployment

Continuous Deployment takes the DevOps maturity one step further by automating the final stage of delivery.

It indicates maturity in the product and the process when confidence in the automation is high enough that no human review is required to push a change straight to Production.

This means that the time from code commit to Production deploy can be very short (less than an hour).



This Photo by Unknown Author is licensed under CC BY-SA

Introduction to Jenkins

Meet Jenkins (1 of 3)

Jenkins is an open-source automation server, used extensively to integrate Continuous Integration and Continuous Delivery practices.

Originally released in 2011, Jenkins is a Java based application developed by an active Open-Source community.

Competitors include Atlassian's Bamboo and Microsoft's Azure DevOps, which are paid products.

With a 50% market share, this open-source tool is a de facto standard in this domain – no small feat!

We will complete several exercises with Jenkins which demonstrate its capabilities, but also more broadly, what to expect from a CICD tool.



This Photo by Unknown Author is licensed under [CC BY-SA](#)

Meet Jenkins (2 of 3)

Strengths

- Plugins: there are over a thousand plugins, which have been created to increase interoperability with any conceivable application server or tooling. Developers also have the option to create their own plugins.
- Its open-source community: Jenkins is updated regularly, keeping on top of security vulnerabilities. e.g., recent upgrades

2.417 2023-08-01
2.416 2023-07-26
2.414 2023-07-11
2.413 2023-07-04
2.412 2023-06-27
2.411 2023-06-20
2.410 2023-06-13
2.409 2023-06-06

- Reliability: From startups to large enterprises, Jenkins is known and trusted, evidenced by its high market share.



This Photo by Unknown Author is licensed under [CC BY-SA](#)

Meet Jenkins (3 of 3)

Challenges

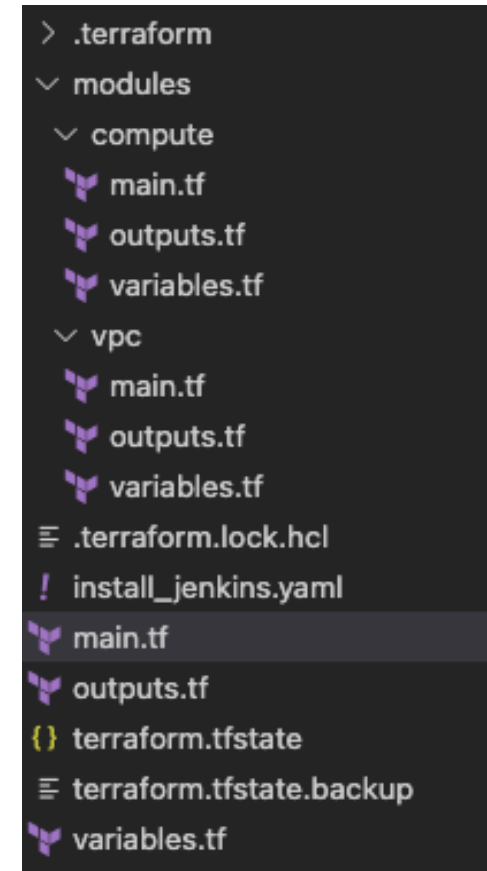
- Cost of Ownership: as it is an open-source project rather than a SaaS offering, there is a significant amount of work to keep Jenkins master nodes up and running with latest versions. CloudBees offer a paid version and other vendors will host Jenkins for you, but neither are as popular as the self-managed version.
- Learning curve: given the openness of the platform, there is more to learn than other solutions that abstract more away from the admin/user. This is also a pro for some consumers.
- Plugin management: interdependencies between plugins mean that upgrades can have unintended consequences with plugins, which highlights the need for a robust upgrade process with sufficient testing included.



This Photo by Unknown Author is licensed under [CC BY-SA](#)

Install Jenkins (1 of 6)

- For this next segment we will install and configure a Jenkins server, using the now familiar Terraform-Ansible format that we have used several times.
- Reviewing the folder structure, we use compute and vpc modules, that are referenced by the main.tf file in the root folder.
- This main.tf also installs Ansible and executes the install_jenkins.yaml playbook, which is run locally on the provisioned server.
- The terraform.tfstate* files and .terraform folder are not included in Github, they are created as part of execution of the playbook.



Install Jenkins (2 of 6)

Points to note

- Remember to update location of your private and public keys in `modules>compute>variables.tf`
- The Jenkins URL is included in the output as it is in `output.tf`, which references `modules>compute>outputs.tf>server_ip`
- The admin password is also included in the output by doing a `cat` of the password file to a variable in Ansible and then debugging that variable out.
- Review the rest of the Ansible file, in particular how Jenkins installation is completed.



```
outputs.tf > output "Jenkins-Public-URL"
1  #-----outputs.tf-----
2  #=====
3  output "Jenkins-Public-URL" {
4    value = "http://${module.compute.server_ip}:8080"
5  }
```



```
modules > compute > outputs.tf > ...
6
7  output "server_ip" {
8    value = aws_instance.jenkins.public_ip
9  }
```



```
47  - name: Get init password Jenkins
48    shell: cat /var/lib/jenkins/secrets/initialAdminPassword
49    changed_when: false
50    register: result
51
52  - name: Print init password Jenkins
53    debug:
54      var: result.stdout
```

Install Jenkins (3 of 6)

Progress through terraform init, plan and apply and review the output. The most important items are:

1. Jenkins admin password which we will use during setup
2. Jenkins URL, which you copy to a browser to complete the configuration of the tool

```
module.compute.aws_instance.jenkins (remote-exec): TASK [Print init password Jenkins]
module.compute.aws_instance.jenkins (remote-exec): ok: [localhost] => {
module.compute.aws_instance.jenkins (remote-exec):   "result.stdout": "90a70d2e4e61
442593f3bf41ddc09e45"
module.compute.aws_instance.jenkins (remote-exec): }

module.compute.aws_instance.jenkins (remote-exec): PLAY RECAP *****
module.compute.aws_instance.jenkins (remote-exec): localhost : ok=10
changed=6    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

module.compute.aws_instance.jenkins: Creation complete after 4m20s [id=i-000a1f3bdbb5
ffca9]

Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:

Jenkins-Public-URL = "http://3.90.36.55:8080"
```


Install Jenkins (4 of 6)

1. Loading the URL and you will be prompted to enter the password.
2. Next click 'Install suggested plugins', where most commonly used plugins are installed including git, gradle and others
3. Next create an Admin user, using your own name and email address. Click through to start using Jenkins.
4. Click on '+ New Item'.

← → ↻ 3.90.36.55:8080/view/all/newJob ☆ ⚙️ 🔔 🔒 🔑

Jenkins 🔍 Search (⌘+K) ?

Dashboard > All >

Enter an item name

» This field cannot be empty, please enter a valid name

-  **Freestyle project**
This is the central feature of Jenkins. Jenkins will build your project, combining any SCM with any build system, and this can be even used for something other than software build.
-  **Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
-  **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
-  **Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
-  **Multibranch Pipeline**
Creates a set of Pipeline projects according to detected branches in one SCM repository.
-  **Organization Folder**
Creates a set of multibranch project subfolders by scanning for repositories.

REST API Jenkins 2.401.3

Install Jenkins (5 of 6)

5. Enter a name for the item, selecting 'Freestyle Project' and clicking OK.
6. Scroll down to Build Steps, skipping everything else.
7. Select 'Execute Shell' from the dropdown.
8. Enter the following in the command window

```
whoami  
ps -ef
```

The screenshot shows the Jenkins configuration page for a Freestyle Project. The breadcrumb navigation at the top reads 'Dashboard > My Freestyle > Configuration'. Below this, there is a section for 'Poll SCM' with an unchecked checkbox and a help icon. The 'Build Environment' section contains several unchecked checkboxes: 'Delete workspace before build starts', 'Use secret text(s) or file(s)', 'Add timestamps to the Console Output', 'Inspect build log for published build scans', 'Terminate a build if it's stuck', and 'With Ant'. The 'Build Steps' section is highlighted with a dashed border and contains a single step named 'Execute shell'. Below the step name, there is a 'Command' field with a text area containing the commands 'whoami' and 'ps -ef'. A link 'See the list of available environment variables' is provided. At the bottom of the configuration page, there are 'Save' and 'Apply' buttons.

Dashboard > My Freestyle > Configuration

☐ Poll SCM ?

Build Environment

☐ Delete workspace before build starts

☐ Use secret text(s) or file(s) ?

☐ Add timestamps to the Console Output

☐ Inspect build log for published build scans

☐ Terminate a build if it's stuck

☐ With Ant ?

Build Steps

≡ Execute shell ?

Command

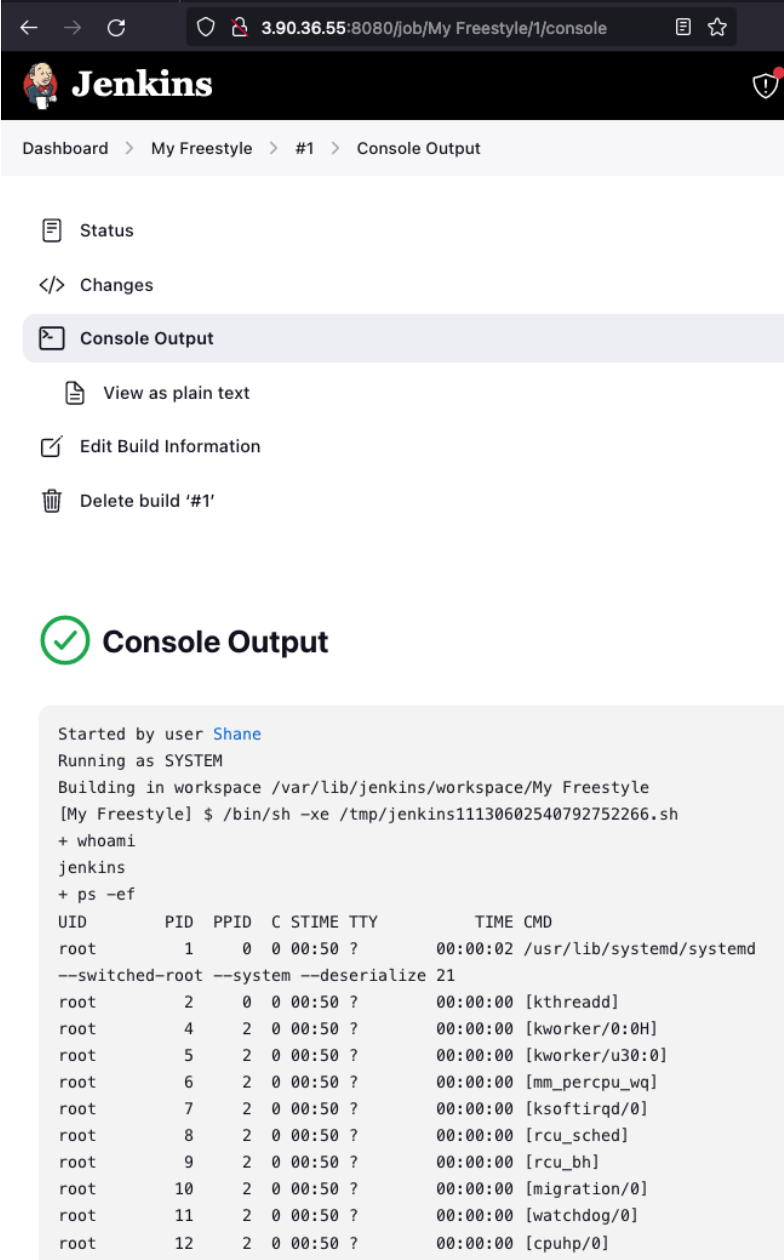
See [the list of available environment variables](#)

```
whoami  
ps -ef
```

Save Apply

Install Jenkins (6 of 6)

9. Click Save and on the next screen, click 'Build Now'.
10. Under 'Build History' click on #1, which is the first invocation of your job.
11. The 'Console Output' is often the most useful option on the next screen, one you will use many times for troubleshooting.
12. Assuming all well there, then congratulations on installing Jenkins and creating your first job.
13. Don't forget terraform destroy.



The screenshot shows the Jenkins web interface. At the top, the address bar displays '3.90.36.55:8080/job/My Freestyle/1/console'. The Jenkins logo is in the top left, and a shield icon is in the top right. The breadcrumb navigation shows 'Dashboard > My Freestyle > #1 > Console Output'. On the left sidebar, there are links for 'Status', 'Changes', 'Console Output' (which is highlighted), 'View as plain text', 'Edit Build Information', and 'Delete build #1'. The main content area has a green checkmark icon and the title 'Console Output'. Below this, the console output text is displayed, showing the user 'Shane' starting the job, running as 'SYSTEM', and building in the workspace. The output includes a shell command to run 'ps -ef' and a table of running processes.

```
Started by user Shane
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/My Freestyle
[My Freestyle] $ /bin/sh -xe /tmp/jenkins11130602540792752266.sh
+ whoami
jenkins
+ ps -ef

```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0	00:50	?	00:00:02	/usr/lib/systemd/systemd
--switched-root --system --deserialize 21							
root	2	0	0	00:50	?	00:00:00	[kthreadd]
root	4	2	0	00:50	?	00:00:00	[kworker/0:0H]
root	5	2	0	00:50	?	00:00:00	[kworker/u30:0]
root	6	2	0	00:50	?	00:00:00	[mm_percpu_wq]
root	7	2	0	00:50	?	00:00:00	[ksoftirqd/0]
root	8	2	0	00:50	?	00:00:00	[rcu_sched]
root	9	2	0	00:50	?	00:00:00	[rcu_bh]
root	10	2	0	00:50	?	00:00:00	[migration/0]
root	11	2	0	00:50	?	00:00:00	[watchdog/0]
root	12	2	0	00:50	?	00:00:00	[cpuhp/0]

Next, Test Automation

Test Automation

Without automated testing, the more code we write, the more time and money is required to test our code—in most cases, this is a totally unscalable business model for any technology organization.

- Gary Gruver

<https://itrevolution.com/articles/case-study-automated-testing-google/>



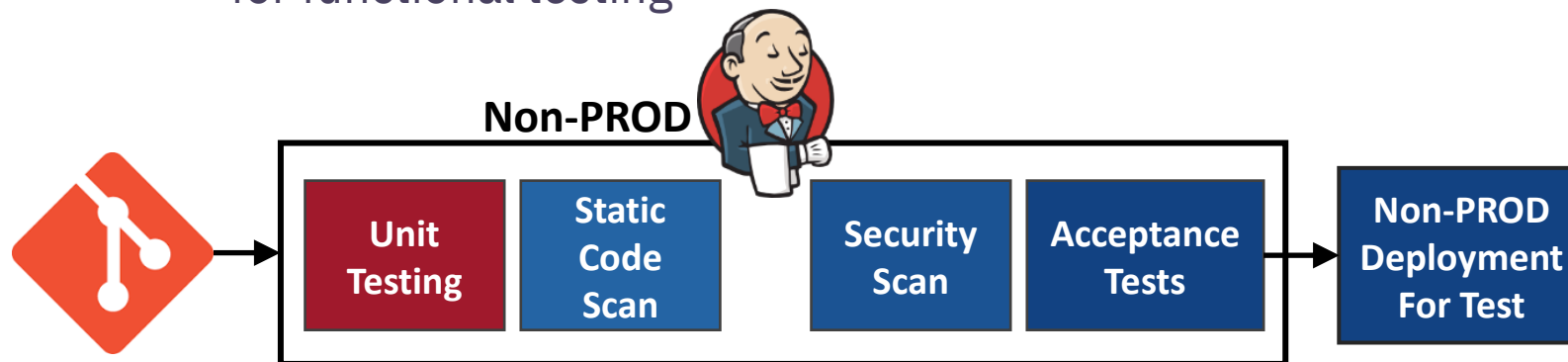
Test Automation (con't)

- In our review of Continuous Integration, it was clear that test automation is an integral part of that practice.
- Test automation needs to be fast and reliable.
- All code checked in to version control should be automatically built and tested in a production like environment.
- The IaC practices we have evaluated support this approach, where test environments can be created to be consistent with production, reinforcing the importance of IaC.



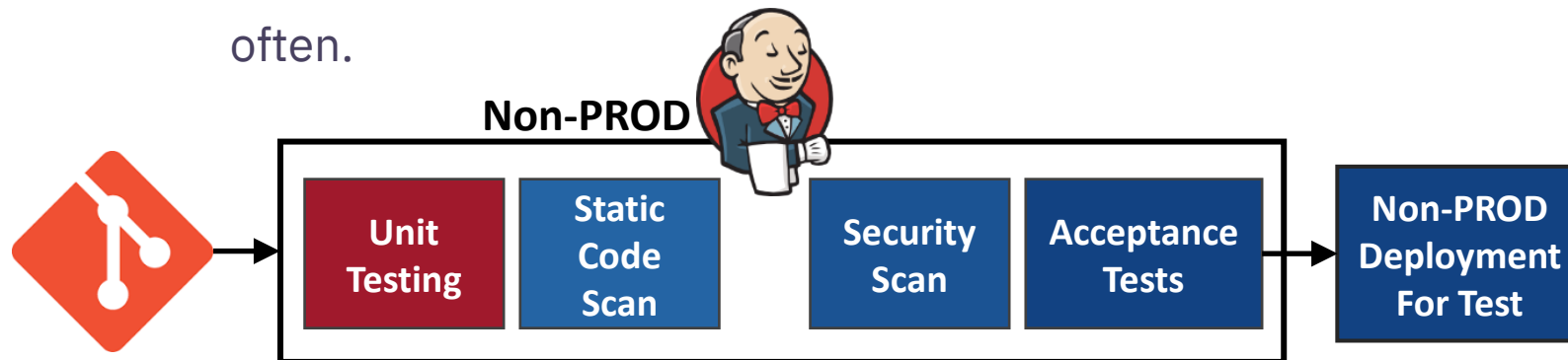
Test Automation – CI Pipelines

- Continuous Integration pipelines bring many strands of test automation together
 - Unit tests: code-based tests created by developers as part of development
 - Static code analysis: a form of testing the code quality e.g., SonarCloud looks for bugs, code smells, duplication, unit test coverage, etc
 - Security scan: check for unsecure coding patterns in code e.g., SQL injection
 - Acceptance tests: end-to-end tests where code is deployed a non-PROD environment for functional testing

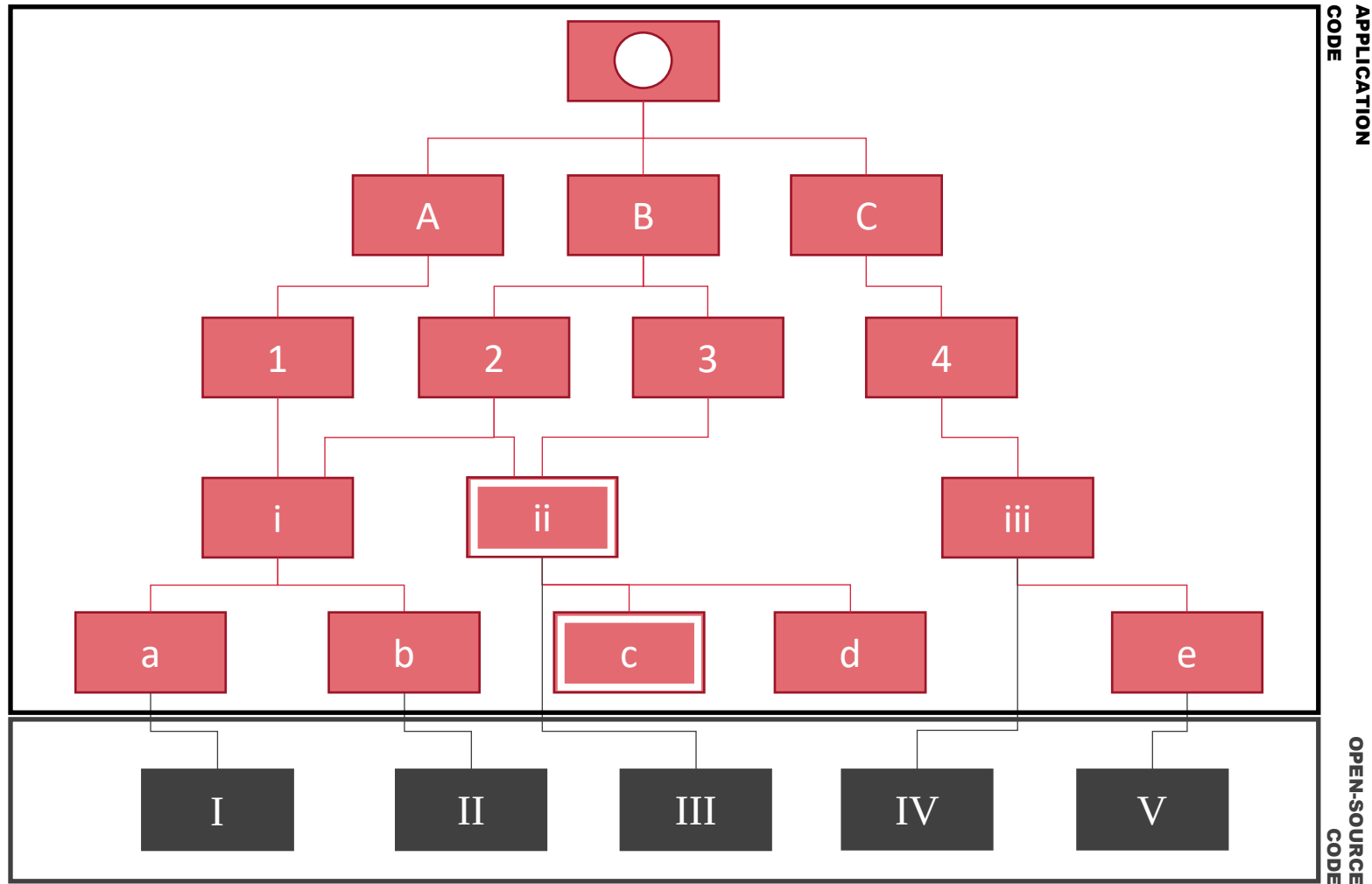


Test Automation – CI Pipelines (con't)

- Continuous Integration pipelines bring many strands of test automation together
 - When each of these test types pass, the code can be packaged and pushed to an artifact management tool such as Sonatype's Nexus or Jfrog's Artifactory
 - With so much validation in the CI pipeline, these artifacts can be release candidates at this point, with full audit trail on test execution and results.
 - If tests fail then it should stop the line, with a team culture of fix the issue before moving on. True CI is enabled by this approach due as small batches can be committed to trunk often.

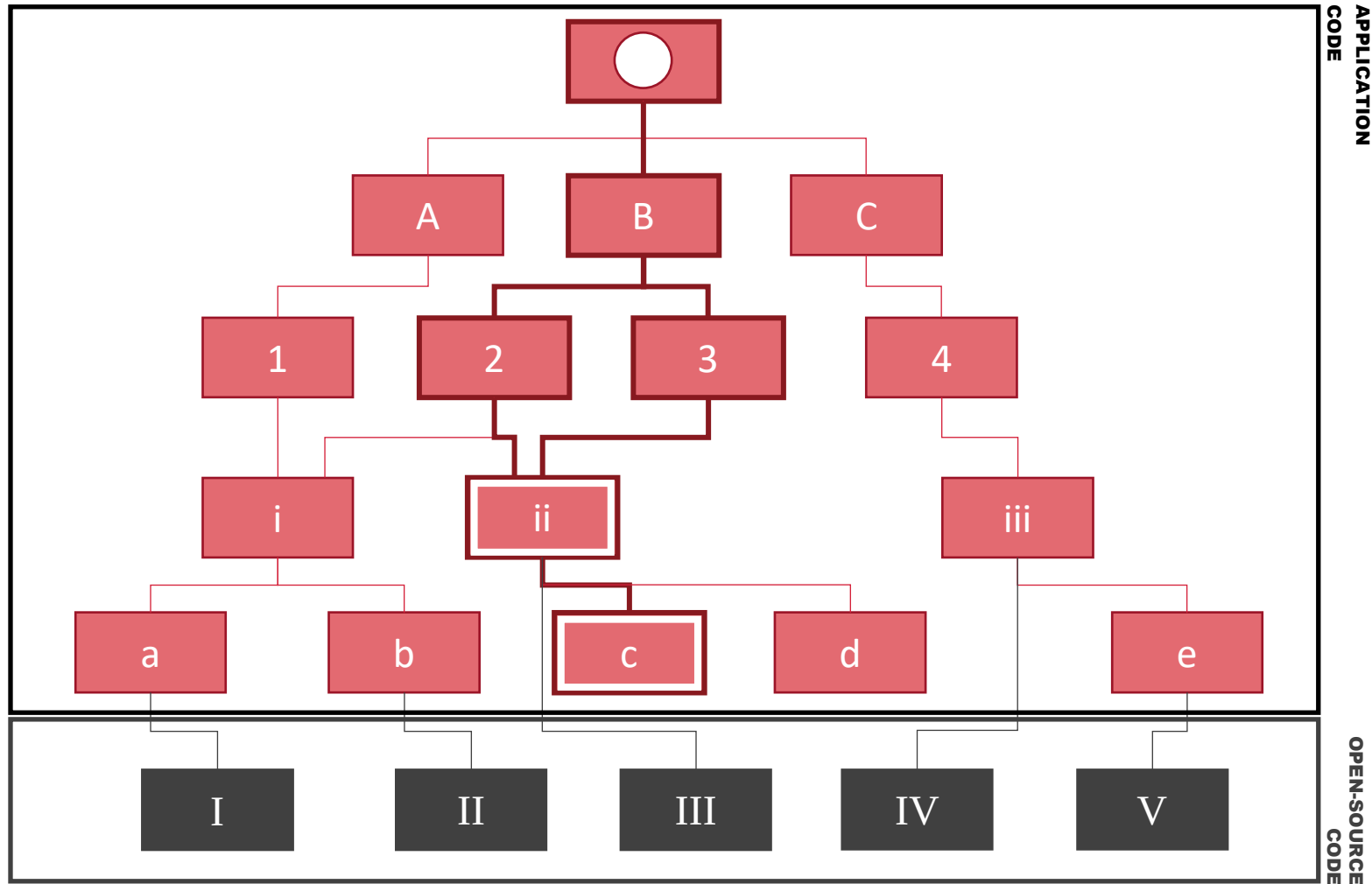


How development works (1 of 4)



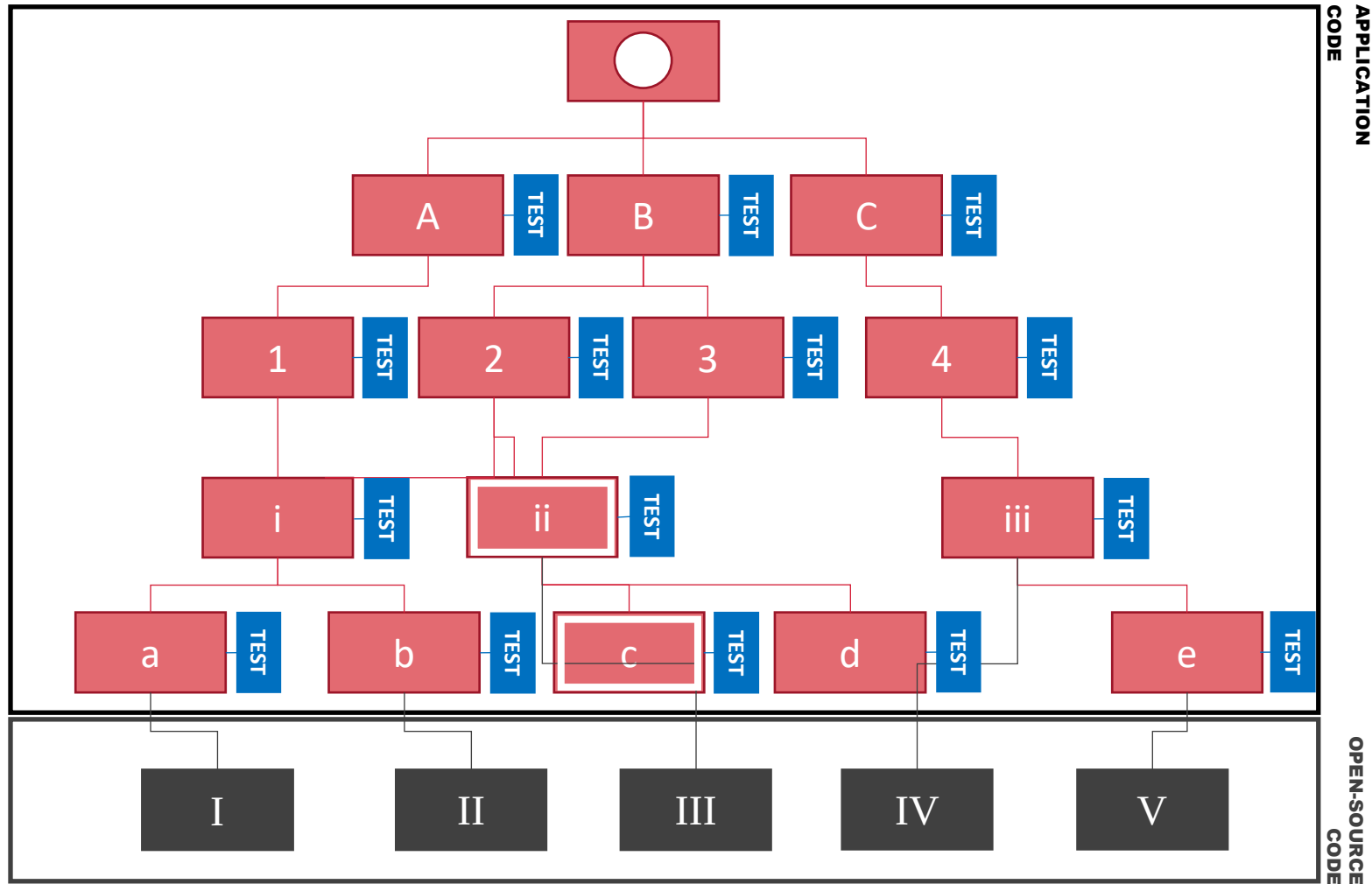
- To understand how these tests improve flow, it helps to understand how the development process works.
- A well-designed application looks like this, with a clear hierarchy of classes, some reuse of classes and no obvious spaghetti code.
- Typically, an application change or enhancement is requested that requires updates to one of more classes or source code files located somewhere in the application codebase.

How development works (2 of 4)



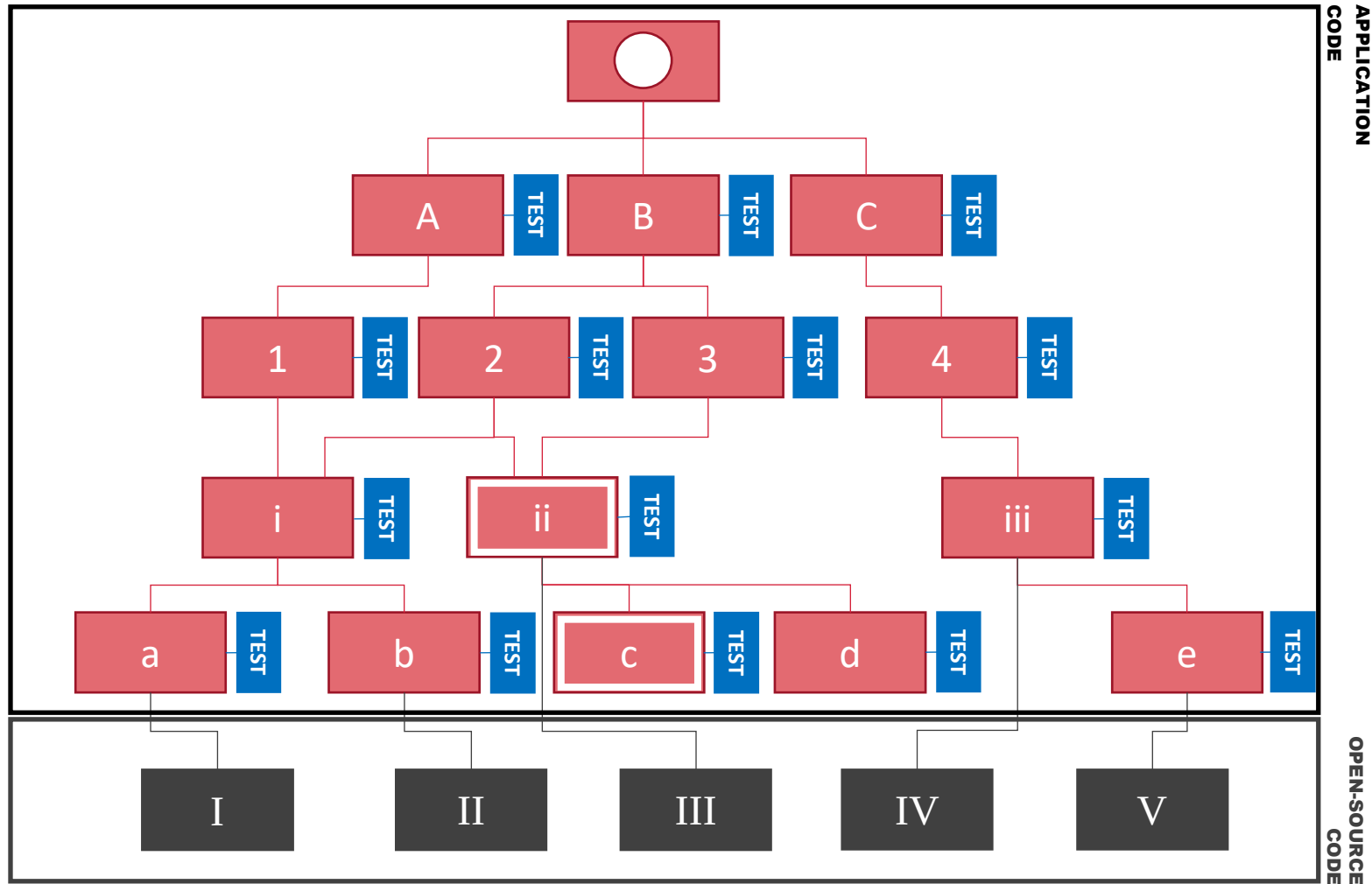
- Navigating to the code that needs to be changed is time consuming but needs to be done by developer and tester.
- There may be more than one path and test data required, or other dependencies may slow progress in getting to the point of making/testing changes.

How development works (3 of 4)



- Another approach is to write unit tests for each class as you change it, so that the class can be tested directly, without the cognitive load of navigating through the application, while also removing other dependencies.
- Over time a test suite of unit tests is created that serves as a very fast running regression pack, executed as one of the first stages in your CI pipeline.
- Code with unit tests is better designed because of the constraints this test approach applies to the code.

How development works (4 of 4)



Benefits

Shift Left.

Automate.

Fast Feedback.

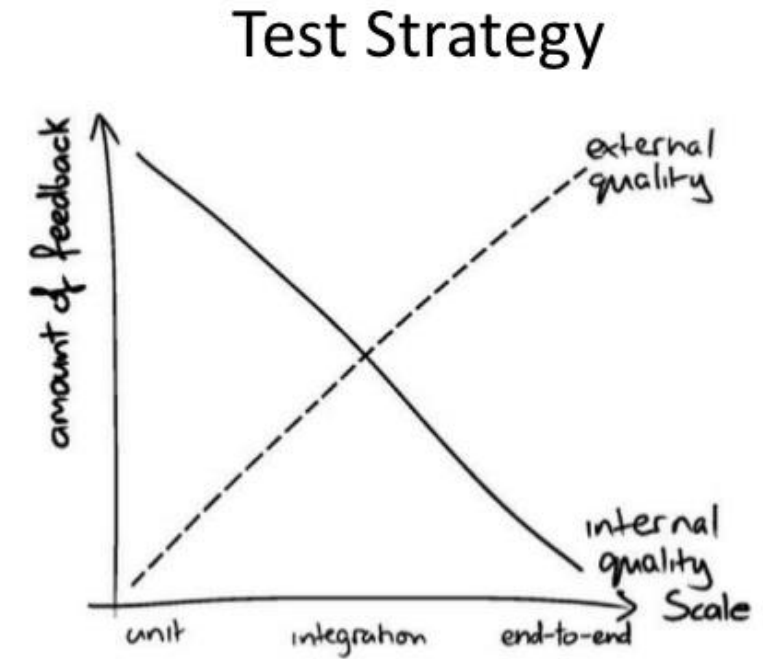
Innovate & Experiment.

Continuous Improvement.

Committed to Technical
Excellence.

Unit Tests

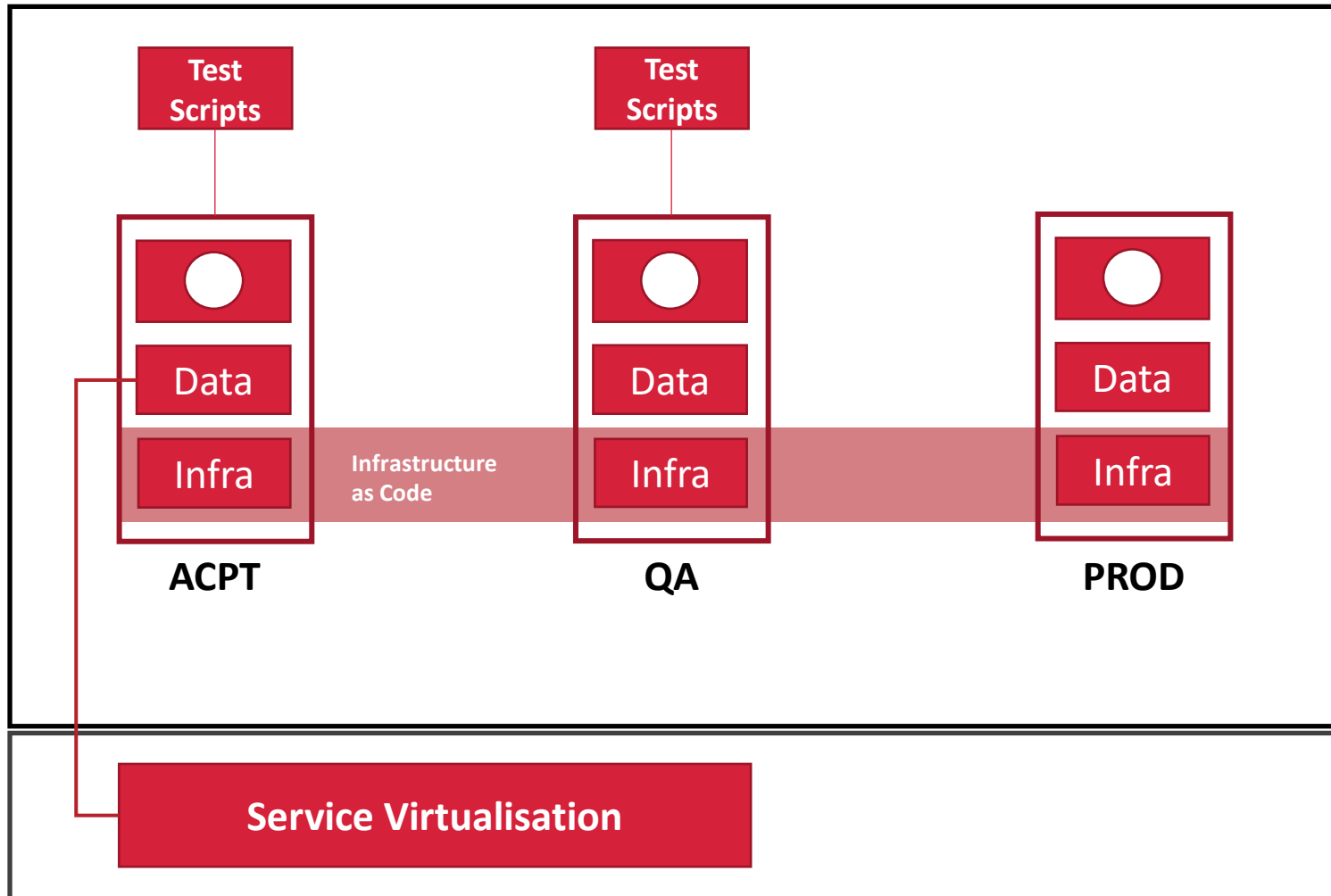
- In writing unit tests, a more structured design emerges.
- There is now an extensive test suite, which runs as part every build maintaining high internal quality.
- This serves as documentation and a safety net for new developers.
- A well-structured application will be maintainable for 10 years. One that is poorly structure becomes unstable after 5-7 years.
- Test Driven Development is a well recognized approach ensures unit testing is part of all development effort.



Taken from *Growing Object-Oriented Software, Guided by Tests*

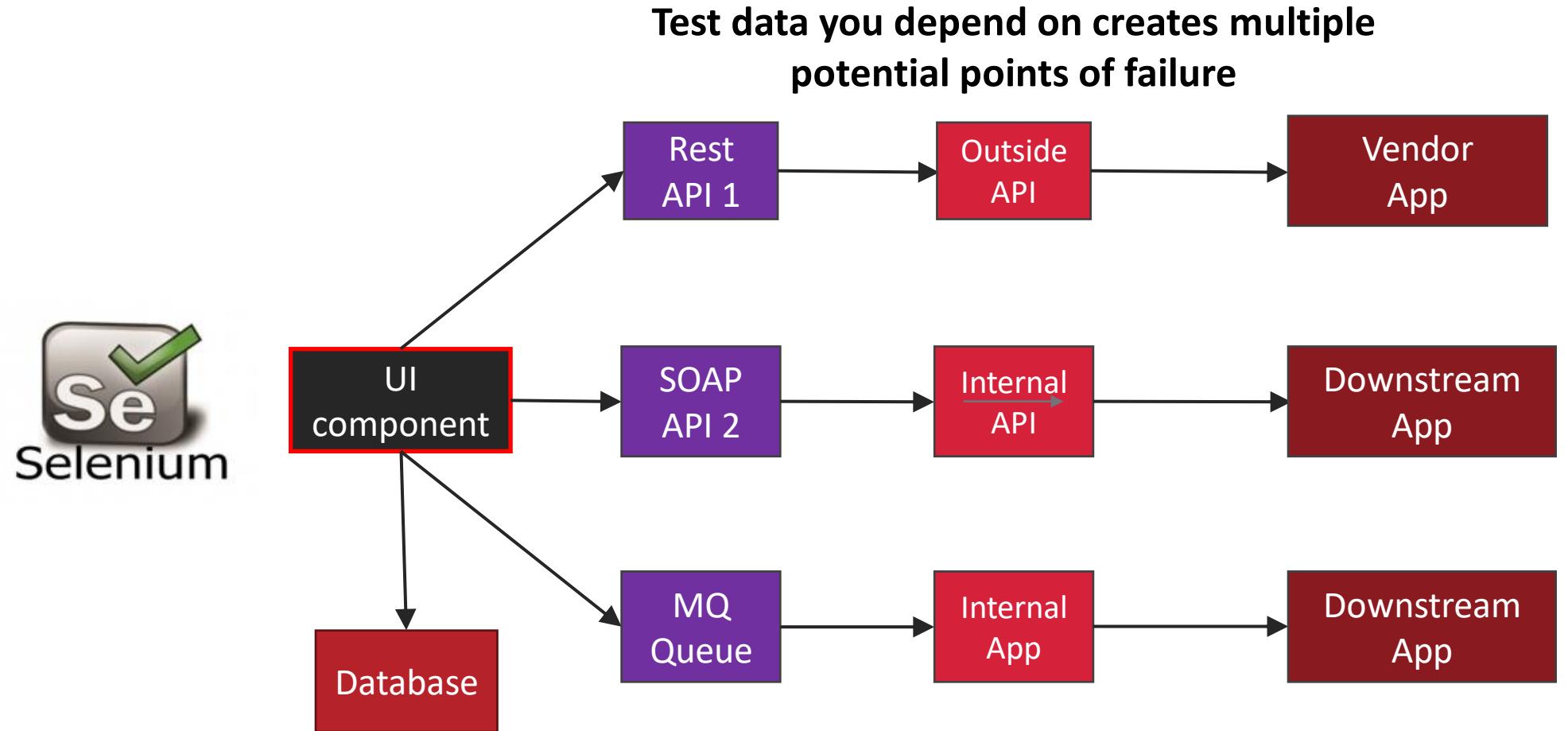
34

How acceptance tests work.



- Tools such as Selenium allow tests to be executed by navigating, entering data and verifying output through web applications.
- Consistency at the infra layer provides confidence that non-PROD tests are representative of PROD scenarios.
- Data which is sourced through SV is stable and prevents false positives in test results.
- Well designed and maintained test scripts are essential to smooth running of this process.

Sample Test Data Dependencies



Add Service Virtualization



Record all request and response data
in SV for one smoke test run.
Play it back for all future smoke tests.

UI
component

SV

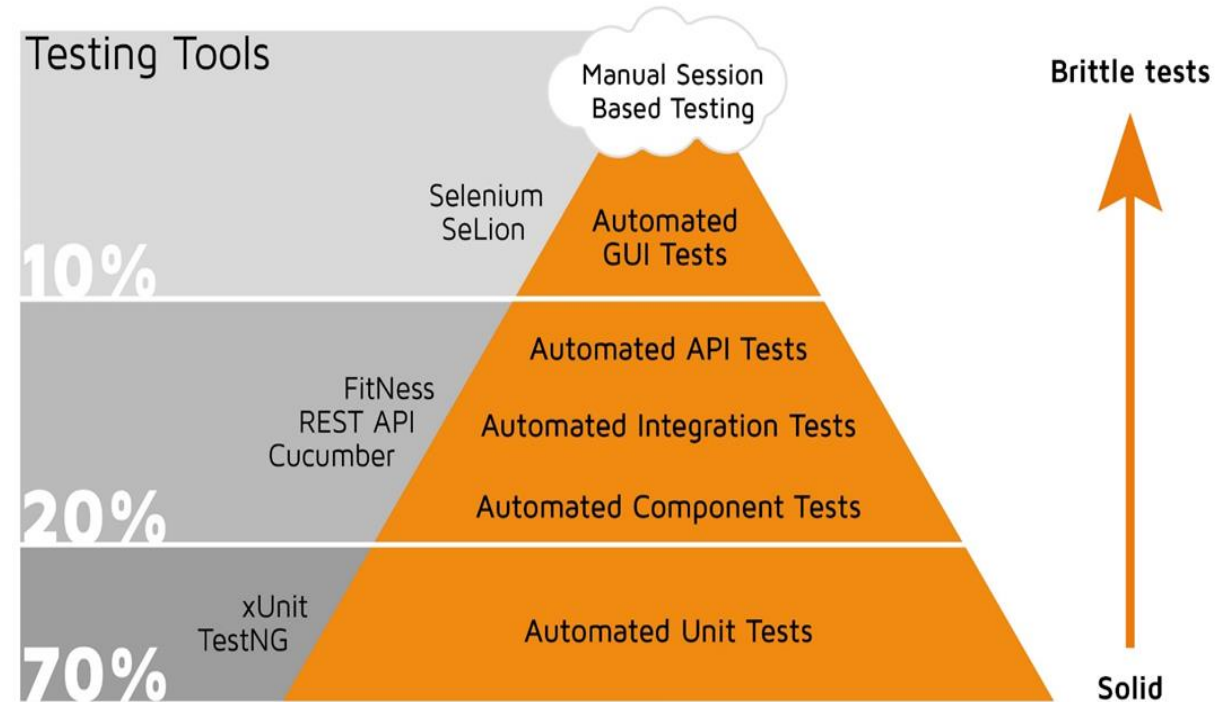
Database

Remove data dependency for all downstream systems,
replacing it with one consistent source of data.

This reduces data noise in test failures and puts the focus
on code quality.

Testing Pyramid

- Unit tests are the bedrock of DevOps test automation.
- With a target time of 10-15 minutes for pipeline execution, unit tests provide very efficient feedback.
- 80%+ unit test coverage is recommended for applications with straight to PROD deployments.



Journey Map

